

**National Taipei University of Technology**  
**Object-Oriented Analysis and Design**  
**Homework #6**

**arch-checker**

A Fluent Architectural Style Compliance Tool for Java

**Team Member:**

113598009 資工碩二 李俊威

## Table of Contents

|                                                                      |    |
|----------------------------------------------------------------------|----|
| 1. Requirement Document .....                                        | 3  |
| 1.1 Change History .....                                             | 3  |
| 1.2 Problem Statement .....                                          | 3  |
| 1.3 Summary of System Features .....                                 | 4  |
| 1.4 Use Case Diagram.....                                            | 4  |
| 1.5 Use Cases .....                                                  | 5  |
| 1.6 Non-Functional Requirements and Constraints .....                | 10 |
| 1.7 Glossary .....                                                   | 11 |
| 1.8 Software Environments.....                                       | 12 |
| 2. Domain Model .....                                                | 12 |
| 2.1 Domain Class Diagram (Concepts) .....                            | 12 |
| 2.2 Associations .....                                               | 14 |
| 2.3 Attributes.....                                                  | 15 |
| 3. Revision Based on TA Comments, Midterm Review, and Findings ..... | 15 |
| 4. Measurement.....                                                  | 16 |

# 1. Requirement Document

## 1.1 Change History

| Version | Date       | Author | Description                                                                                                                                                                                                                                            |
|---------|------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0     | 2026-03-03 | 李俊威    | Initial submission (HW1) — Problem Statement                                                                                                                                                                                                           |
| 2.0     | 2026-03-12 |        | Add System Features, Use Cases, NFR, Glossary (HW2)                                                                                                                                                                                                    |
| 3.0     | 2026-04-01 |        | Add Domain Model; revise HW2: condense Problem Statement, rename SF-XX to FEA-XX, remove CLI commands from Use Cases, generalize error messages, remove implementation details                                                                         |
| 4.0     | 2026-04-17 |        | Add Design (Logical Architecture, Use-Case Realizations with GRASP, Design Class Model); revise HW3: remove non-system-state items from UC-02/UC-03 preconditions, reformat Domain Model descriptions as tables                                        |
| 5.0     | 2026-04-22 |        | Revise HW4: compact UC-01/UC-04 Sequence Diagrams, redraw Design Class Diagram, add Class Specifications table                                                                                                                                         |
| 6.0     | 2026-04-29 |        | 新增 Implementation Class Model、Programming、Unit Testing 三章節（HW5）；系統以 Maven 專案交付，使用 JavaParser、SnakeYAML、picocli、JUnit 5。                                                                                                                                |
| 7.0     | 2026-05-05 |        | 重新命名介面與類別以更精確反映職責：ProfileRepository→ProfileLoader、SuppressionRepository→SuppressionStore、ArchitectureConstraint→ComplianceRule（套件 domain.constraint→domain.rule）；同步更新 Implementation Class Diagram、Sequence Diagrams 與內文敘述。                            |
| 8.0     | 2026-05-26 |        | Iteration II：選定 UC-02 Define Style Profile 與 UC-03 Check with Interactive Fix Suggestions 為本迭代要實作之 use cases；於 Domain Model 新增 Violation.suggestion [0..1] 屬性以支援 UC-03 顯示修正建議；依 HW6 規定移除 Design／Implementation／Programming／Unit Testing 四章節，留待 HW7 提交。 |

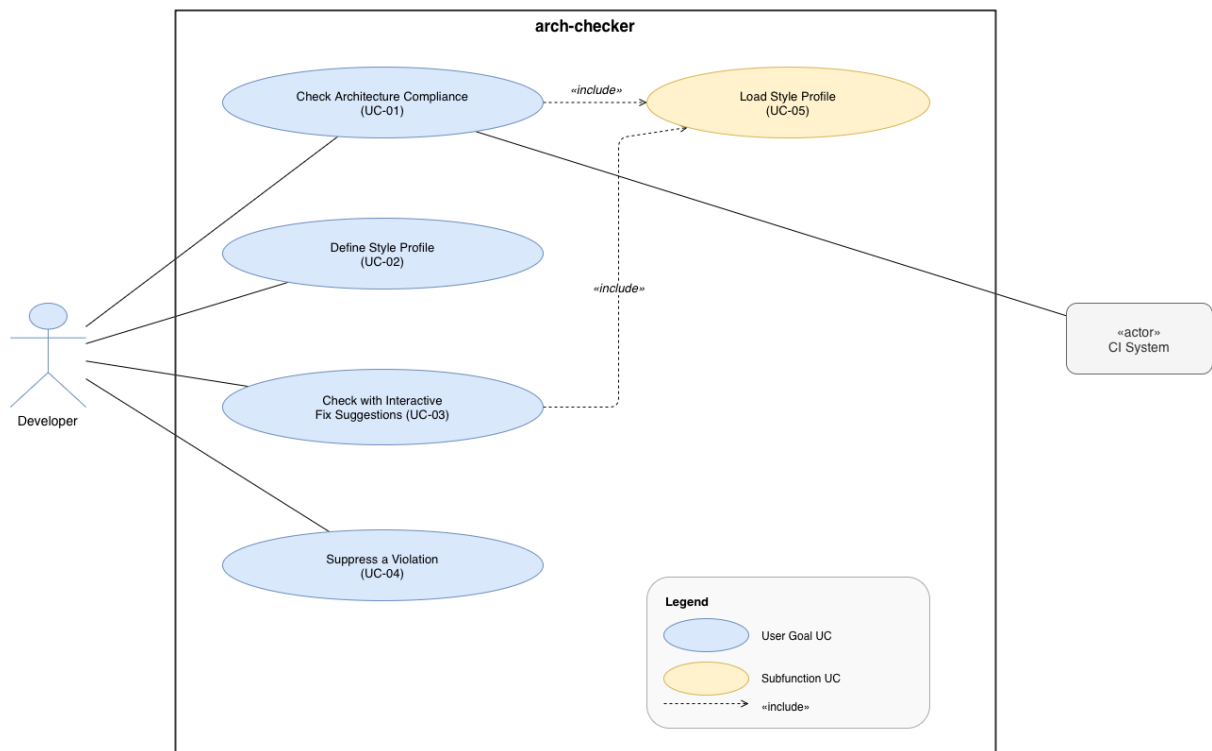
## 1.2 Problem Statement

隨著軟體系統規模擴大與 AI 輔助程式碼生成的普及，人工驗證架構風格合規性日益困難，成為開發週期的瓶頸。現有工具（如 ArchUnit）雖可將架構限制寫成測試規則，但使用門檻較高，開發者需熟悉底層 API 並撰寫大量樣板程式碼，缺乏以風格為中心的高階抽象。arch-checker 是一套專為 Java 開發者設計的命令列工具，用於自動驗證 Java 專案的架構風格合規性。使用者可透過宣告式 API 以 YAML 定義架構風格設定檔（Style Profile），將命名、依賴、繼承等規則封裝為可重複使用的風格單元。系統透過 Java 原始碼的 AST 解析，精確偵測結構性違規，並產出含檔案路徑、行號與錯誤描述的完整報告，適用於本地開發、程式碼審查與持續整合情境。

### 1.3 Summary of System Features

| ID     | Feature Name                      | Priority | Description                                                                             |
|--------|-----------------------------------|----------|-----------------------------------------------------------------------------------------|
| FEA-01 | Architecture Compliance Checking  | High     | 掃描 Java 專案目錄，依使用者定義的 Style Profile 偵測違規                                                 |
| FEA-02 | Style Profile Definition via YAML | High     | 使用者以 YAML 格式定義任意架構風格規則（DependencyRule、NamingRule、SupertypeRule、PackageRule）；工具不預設任何內建風格 |
| FEA-03 | Violation Report Generation       | High     | 輸出含檔案路徑、行號、規則 ID、說明的違規報告                                                                |
| FEA-04 | Multiple Report Formats           | Low      | 支援 Console 與 JSON 兩種輸出格式                                                                |
| FEA-05 | Violation Suppression             | Medium   | 標記特定違規為「已知且接受」，下次不再回報                                                                   |

### 1.4 Use Case Diagram



### Actors:

Developer (Primary)：使用工具定義架構規則並檢查 Java 專案的開發者

CI System (Secondary)：在 CI/CD pipeline 中自動觸發合規檢查，依賴 exit code 判斷結果

| UC    | Name                                   | Level       | Format        | Triggered by            |
|-------|----------------------------------------|-------------|---------------|-------------------------|
| UC-01 | Check Architecture Compliance          | User Goal   | Fully Dressed | Developer / CI System   |
| UC-02 | Define Style Profile                   | User Goal   | Fully Dressed | Developer               |
| UC-03 | Check with Interactive Fix Suggestions | User Goal   | Fully Dressed | Developer               |
| UC-04 | Suppress a Violation                   | User Goal   | Brief         | Developer               |
| UC-05 | Load Style Profile                     | Subfunction | Brief         | UC-01 / UC-03 (include) |

## 1.5 Use Cases

### UC-01: Check Architecture Compliance (Fully Dressed)

|               |                               |
|---------------|-------------------------------|
| Use Case      | UC-01                         |
| Use Case Name | Check Architecture Compliance |
| Scope         | arch-checker CLI              |

|                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Level</b>                      | User Goal                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Primary Actor</b>              | Developer                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Stakeholders and Interests</b> | <p>Developer：想要取得準確且詳細的違規報告，包含檔案路徑與行號，以便快速找出不符合規範的類別，避免費時的人工審查。</p> <p>Team Lead / Architect：想確保整個程式碼庫符合團隊定義的架構規則，零違規代表架構一致性獲得維護。</p> <p>CI System（次要）：想要可靠的 POSIX exit code（0/1/2）來判斷 pipeline 是否通過，不需要解讀報告內文。</p>                                                                                                                                                                                                                                                                                                                                    |
| <b>Preconditions</b>              | <ol style="list-style-type: none"> <li>1. arch-checker 已安裝並可在系統中存取</li> <li>2. 目標 Java 專案目錄存在且可讀取</li> <li>3. Style Profile YAML 檔案存在且語法正確</li> </ol>                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Success Guarantee</b>          | 系統產生完整的 ViolationReport，包含：已檢查檔案數、違規清單、suppressedCount；並回傳 POSIX exit code（0 = pass，1 = violations found）                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Main Success Scenario</b>      | <ol style="list-style-type: none"> <li>1. Developer 發出對指定專案目錄進行架構合規檢查的要求，並指定要使用的 Style Profile</li> <li>2. 系統確認專案目錄存在</li> <li>3. 系統讀取並解析指定的 Style Profile YAML 檔案</li> <li>4. 系統遞迴掃描專案目錄下所有 .java 檔案</li> <li>5. 系統以 JavaParser 將每個 .java 檔解析為 AST（CompilationUnit）</li> <li>6. 系統對每個 CompilationUnit 套用 Profile 中所有 ComplianceRule 規則</li> <li>7. 系統收集所有 Violation，每筆包含：檔案路徑、行號、規則 ID、說明文字</li> <li>8. 系統過濾已被 suppress 的 Violation</li> <li>9. 系統以 Console 格式輸出 ViolationReport（含違規清單與 summary）</li> <li>10. 系統回傳 exit code：無違規則 0，有違規則 1</li> </ol> |
| <b>Extensions</b>                 | <ol style="list-style-type: none"> <li>2a. 專案目錄不存在：系統輸出錯誤訊息，exit code 2，終止</li> <li>3a. Style Profile 檔案不存在：系統輸出錯誤訊息，exit code 2，終止</li> <li>3b. YAML 語法錯誤：系統輸出 YAML 解析錯誤與行號，exit code 2，終止</li> <li>3c. YAML 語法正確但規則定義無效（如未知的 rule type）：系統輸出 validation 錯誤清單，exit code 2，終止</li> <li>4a. 專案目錄下無 .java 檔：系統輸出警告訊息，exit code 0，終止</li> </ol>                                                                                                                                                                                                           |

|                                            |                                                                                                                                                                                                  |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Special Requirements</b>                | 5a. 某 .java 檔語法錯誤無法解析：系統記錄警告訊息，跳過該檔案，繼續處理其餘檔案<br>8a. 所有 Violation 皆已被 suppress：報告顯示全部通過（含已抑制數量），exit code 0                                                                                      |
|                                            | NFR-01（Performance）：掃描 1,000 個 .java 檔案必須在 10 秒內完成（一般開發機）<br>NFR-02（Usability）：所有錯誤訊息必須包含：錯誤原因、位置（檔案/行號）、建議修正動作<br>NFR-05（CI Compatibility）：exit code 遵循 POSIX 標準，可直接整合 GitHub Actions / Jenkins |
|                                            | *a. Style Profile 接受任意 YAML 檔案路徑，不支援內建名稱<br>*b. 輸出格式可切換為 JSON，預設為 Console 文字格式<br>4a. Java 檔案掃描採遞迴深度優先走訪                                                                                         |
| <b>Technology and Data Variations List</b> |                                                                                                                                                                                                  |
| <b>Frequency of Occurrence</b>             | CI 環境：每次 push 觸發，每日可達數十次；本地開發：每小時數次                                                                                                                                                              |
| <b>Miscellaneous</b>                       | 是否需支援 multi-module Maven 專案（多個 src/main/java 子目錄）？<br>大型專案（>10,000 個 .java 檔）是否需要增量掃描機制？                                                                                                         |

## UC-02: Define Style Profile (Fully Dressed) ← Iteration II（本迭代實作）

|                                   |                                                                                                                                  |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case</b>                   | UC-02                                                                                                                            |
| <b>Use Case Name</b>              | Define Style Profile                                                                                                             |
| <b>Scope</b>                      | arch-checker CLI                                                                                                                 |
| <b>Level</b>                      | User Goal                                                                                                                        |
| <b>Primary Actor</b>              | Developer                                                                                                                        |
| <b>Stakeholders and Interests</b> | Developer：想用最少工夫定義出可重複使用的架構規則，希望工具提供模板降低手寫 YAML 的門檻，並在儲存前即時得知格式是否正確。<br>Team Lead：想要團隊共享一份統一的 Profile 並版本控制於 Git，確保規則描述清楚且長期可維護。 |
| <b>Preconditions</b>              | arch-checker 已安裝並可在系統中存取                                                                                                         |
| <b>Success Guarantee</b>          | 一個語法正確、至少含 1 條規則的 .yaml Profile 檔案存在於磁碟，且可被 UC-01 成功載入使用                                                                         |
| <b>Main Success Scenario</b>      | 1. Developer 發出建立新 Style Profile 模板的要求，並指定輸出位置                                                                                   |

|                                            |                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                            | <p>2. 系統產生含預設結構與欄位說明註解的空白 YAML 模板，寫入指定位置</p> <p>3. Developer 完成 YAML 編輯後，發出驗證 Style Profile 的要求</p> <p>4. 系統讀取並解析 YAML 檔</p> <p>5. 系統驗證每條規則的 type (NamingRule / DependencyRule / SupertypeRule / PackageRule) 與所有必填欄位</p> <p>6. 系統驗證每個 package pattern 的語法 (只允許英數字、*、.)</p> <p>7. 系統輸出驗證成功訊息，顯示載入的規則數量</p> <p>8. Developer 在執行 UC-01 時指定此 Profile</p> |
| <b>Extensions</b>                          | <p>3a. Developer 跳過模板建立步驟，直接對已存在的 YAML 發出驗證要求：從 Step 4 開始，正常執行</p> <p>4a. 指定檔案不存在：系統輸出錯誤訊息，終止</p> <p>5a. YAML 頂層結構錯誤 (不是合法的 YAML mapping)：系統輸出錯誤訊息與錯誤行號，終止</p> <p>5b. 遇到未知的 rule type：系統輸出錯誤訊息，列出允許的規則類型，終止</p> <p>6a. package pattern 含非法字元：系統輸出錯誤訊息，終止</p> <p>7a. YAML 格式正確但 rules 清單為空：系統輸出警告訊息，表示 Profile 有效但不會產生違規，exit code 0</p>               |
| <b>Special Requirements</b>                | <p>NFR-02 (Usability)：所有 validation 錯誤訊息必須清楚指出 YAML 錯誤行號，並提供修正建議</p> <p>NFR-04 (Extensibility)：擴充新的規則類型不應影響現有規則的合規檢查行為</p>                                                                                                                                                                                                                            |
| <b>Technology and Data Variations List</b> | <p>*a. Profile 格式為 YAML (以 SnakeYAML 解析)，採 UTF-8 編碼</p> <p>*b. Developer 可使用任意文字編輯器 (VS Code、IntelliJ、Vim 等)，不需 IDE plugin</p> <p>2a. 若輸出目錄不存在，系統自動建立</p>                                                                                                                                                                                             |
| <b>Frequency of Occurrence</b>             | <p>每個新專案通常執行一次；規則有異動時重複執行 validate；相對低頻，以週或月為單位</p>                                                                                                                                                                                                                                                                                                   |
| <b>Miscellaneous</b>                       | <p>是否提供 JSON Schema 供 IntelliJ / VS Code 在編輯 Profile YAML 時自動補全？</p> <p>Profile 是否支援繼承 (extends: &lt;other-yaml-path&gt;) 以在既有規則上擴充？</p>                                                                                                                                                                                                              |

### UC-03: Check Architecture Compliance with Interactive Fix Suggestions (Fully



Dressed) ← Iteration II (本迭代實作)

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case                   | UC-03                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Use Case Name              | Check Architecture Compliance with Interactive Fix Suggestions                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Scope                      | arch-checker CLI                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Level                      | User Goal                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Primary Actor              | Developer                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Stakeholders and Interests | <p>Developer：想逐條審視每個 Violation 並當場決定是否 suppress，希望系統提供可操作的修復建議，而不只是列出錯誤清單。</p> <p>Team Lead：想知道哪些 Violation 是「已知且接受」的技術債，並有持久化且可稽核的 suppression 紀錄，而非無聲地被略過。</p>                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Preconditions              | <ol style="list-style-type: none"> <li>1. 同 UC-01 的前置條件</li> <li>2. Terminal 支援互動式輸入（非 pipe 重定向的 non-TTY 模式）</li> </ol>                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Success Guarantee          | 對每條 Violation，三種結果之一成立：(a) Suppression 記錄寫入 .arch-checker-suppress.yaml，(b) Violation 保留在報告中，(c) 流程中止；最終 ComplianceReport 反映所有決策結果                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Main Success Scenario      | <ol style="list-style-type: none"> <li>1. Developer 發出以互動模式對指定專案進行架構合規檢查的要求，並指定 Style Profile</li> <li>2. 系統完成掃描流程（同 UC-01 Step 2–7）：走訪檔案、解析 AST、套用規則、收集所有 Violation</li> <li>3. 系統展示第一條 Violation，顯示：檔案路徑與行號、違反的 ComplianceRule ID 與說明、RefactoringSuggestion（若存在）</li> <li>4. 系統提示是否接受 suppression?</li> <li>5. Developer 輸入 y：系統將此 Violation 的 rule ID 與時間戳記寫入 .arch-checker-suppress.yaml</li> <li>6. 系統展示下一條 Violation（回到 Step 3）</li> <li>7. 所有 Violation 處理完畢後，系統輸出最終 ComplianceReport（已扣除接受的 suppression）</li> <li>8. 系統回傳 exit code（0 = 無未接受 Violation，1 = 有未接受 Violation）</li> </ol> |
| Extensions                 | <p>3a. 此 Violation 有 RefactoringSuggestion：系統在顯示 [y/n/q] 提示前，額外顯示建議操作</p> <p>3b. 掃描後無 Violation：系統直接輸出全部通過訊息，不進入互動迴圈，exit code 0</p> <p>5a. Developer 輸入 n：此 Violation 保留在最終報告中，系統繼</p>                                                                                                                                                                                                                                                                                                                                                                                                       |

|                                            |                                                                                                                                                                                                                                                             |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                            | <p>續下一條（Step 6）</p> <p>5b. Developer 輸入 q：系統中止互動迴圈，輸出已處理部分的 partial report，exit code 1</p> <p>5c. Developer 輸入非法值（非 y/n/q）：系統輸出 Invalid input. Please enter y, n, or q:，重新等待輸入</p> <p>8a. 所有 Violation 均被 suppress（全部輸入 y）：報告顯示全部通過（含已抑制數量），exit code 0</p> |
| <b>Special Requirements</b>                | <p>NFR-02（Usability）：互動提示須清晰易懂，Developer 無需查閱文件即能理解 [y/n/q] 各選項的含義</p> <p>NFR-06（Testability）：互動流程的合規性檢查邏輯必須可透過 unit test 獨立驗證，不依賴完整的 CLI 環境</p>                                                                                                            |
| <b>Technology and Data Variations List</b> | <p>*a. Suppression 記錄寫入 .arch-checker-suppress.yaml，包含：rule ID、reason（預設為 suppressed interactively）、timestamp</p> <p>4a. [y/n/q] 輸入大小寫不敏感（Y、Yes 均視為接受）</p> <p>*b. 若 stdout 被 pipe 重定向（non-TTY），系統自動降級為非互動批次模式並輸出警告</p>                                      |
| <b>Frequency of Occurrence</b>             | 本地開發初次建立 suppression 清單時使用，通常每個新專案執行一次；後續違規出現時按需使用，整體低頻                                                                                                                                                                                                     |
| <b>Miscellaneous</b>                       | <p>是否需要提供 --accept-all 選項，一次 suppress 所有 Violation？</p> <p>Suppression 是否應設有效期限，避免長期堆積技術債？</p>                                                                                                                                                              |

#### UC-04: Suppress a Violation (Brief)

Developer 發出抑制特定違規的要求，並提供 rule ID 與原因說明。系統將 suppression 記錄（rule ID、reason、timestamp）寫入 .arch-checker-suppress.yaml。下次執行 UC-01 時，於 Step 8 過濾生效。

#### UC-05: Load Style Profile (Brief — Subfunction)

UC-01 與 UC-03 的 <<include>> 子流程。系統讀取使用者指定的 Style Profile YAML 檔，解析並驗證規則定義。成功後將 Profile 載入記憶體，供各 rule checker 使用。

### 1.6 Non-Functional Requirements and Constraints

| ID     | Category    | Requirement                                                      |
|--------|-------------|------------------------------------------------------------------|
| NFR-01 | Performance | 完整的 compliance check（UC-01）掃描 1,000 個 .java 檔案必須在 10 秒內完成（一般開發機） |
| NFR-02 | Usability   | 所有錯誤訊息必須包含：錯誤原因、位置（檔案路徑 / 行號）、建議修正動作                             |

| ID     | Category         | Requirement                                                                     |
|--------|------------------|---------------------------------------------------------------------------------|
| NFR-03 | Portability      | 工具需在 JDK 17+ 環境（Windows / macOS / Linux）上運行，不依賴特定 IDE                           |
| NFR-04 | Extensibility    | 系統應支援在不修改既有合規檢查流程的前提下，擴充對新架構規則類型的支援                                             |
| NFR-05 | CI Compatibility | exit code 遵循 POSIX 標準（0 = success，非 0 = failure），可直接整合 GitHub Actions 與 Jenkins |
| NFR-06 | Testability      | 每個架構規則的合規性檢查邏輯必須可透過 unit test 獨立驗證，不依賴完整的 CLI 環境                                |

## 1.7 Glossary

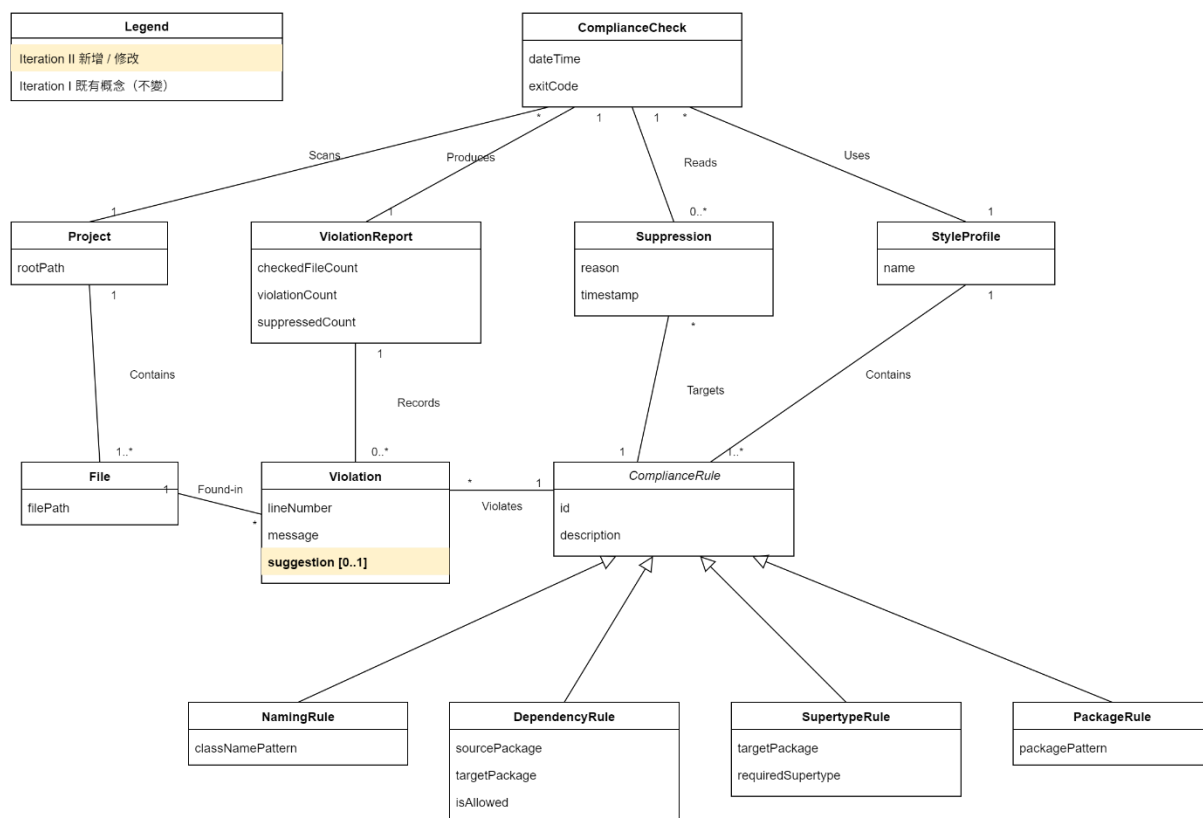
| Term                       | Definition                                                                                                                               |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Style Profile              | 使用者以 YAML 格式定義的架構規則集合，描述特定專案的架構約束；工具本身不提供任何內建 Profile                                                                                    |
| ComplianceRule             | Style Profile 中的單條約束規則，為 NamingRule、DependencyRule、SupertypeRule、PackageRule 的抽象父類別                                                      |
| SupertypeRule              | 一種 ComplianceRule，要求特定 package 內的 class 必須 implement 指定 interface 或 extend 指定 class；以 simple name 比對，不進行 fully qualified name resolution |
| Violation                  | 一筆架構違規記錄，包含：檔案路徑、行號、規則 ID、說明文字                                                                                                           |
| ViolationReport            | 單次 check 執行的完整輸出，彙整所有 Violation 與 summary                                                                                                |
| AST (Abstract Syntax Tree) | Java 原始碼的抽象語法樹表示，由 JavaParser 解析產生，供各 ConstraintChecker 進行分析                                                                             |
| CompilationUnit            | JavaParser 解析單一 .java 檔案後產生的根節點物件                                                                                                        |
| Suppression                | 開發者明確標記「此 Violation 已知且接受」的機制，使其在後續報告中不再出現                                                                                               |
| exit code                  | CLI 程式結束時回傳給 shell 的整數碼（0 = pass，1 = violations found，2 = error）                                                                         |
| Package Pattern            | 規則定義中用來比對 Java package 名稱的萬用字元字串（如 *.ui.*）；只允許英數字、*、.                                                                                    |
| suggestion                 | 針對單筆 Violation 的修正建議文字，於互動模式（UC-03）顯示給開發者；為 Violation 的可選屬性（[0..1]），非獨立概念類別。對應 NFR-02 之「建議修正動作」。                                         |

## 1.8 Software Environments

| Item                 | Choice            | Rationale                         |
|----------------------|-------------------|-----------------------------------|
| Development Language | Java 17 (JDK 17+) | LTS 版本穩定                          |
| AST Parser           | JavaParser 3.x    | 純 Java，API 直覺，無需 JDK tools        |
| CLI Framework        | picocli           | 輕量，annotation-driven，與 Java 17 相容 |
| Profile Format       | YAML (SnakeYAML)  | 可讀性高，易於編輯                         |
| Test Framework       | JUnit 5           | unit testing                      |
| Build Tool           | Maven             | 依賴管理方便                            |

## 2. Domain Model

### 2.1 Domain Class Diagram (Concepts)



上圖為 arch-checker 的 Domain Class Diagram。識別出的概念類別如下：

已採納的概念類別與理由：

| Concept                | Rationale for Inclusion                                           |
|------------------------|-------------------------------------------------------------------|
| <b>ComplianceCheck</b> | 代表一次完整的架構合規檢查作業，對應 UC-01/UC-03 中 Developer 對專案發出檢查要求的動作。系統需記錄每次檢查 |

| Concept                                                                      | Rationale for Inclusion                                                             |
|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|                                                                              | 的時間與結果。                                                                             |
| <b>StyleProfile</b>                                                          | 使用者以 YAML 定義的架構規則集合，包含多條 ComplianceRule。系統在 UC-01 載入 Profile 後，於 UC-02 逐條驗證其中的約束規則。 |
| <b>ComplianceRule</b>                                                        | StyleProfile 中的單條約束規則（抽象概念），為 Profile 的組成元素。設為抽象類別，因其具有四個具體子類型，各自對應不同的架構約束面向。       |
| <b>NamingRule /<br/>DependencyRule /<br/>SupertypeRule /<br/>PackageRule</b> | ComplianceRule 的四種具體子類型，分別檢查類別命名慣例、套件間依賴方向、繼承關係要求與套件結構規範。UC-02 驗證流程中依規則類型分派不同的檢查邏輯。 |
| <b>Project</b>                                                               | 被檢查的目標 Java 專案，UC-01 中 Developer 指定的掃描對象。系統需知道專案路徑以定位其中的 File。                      |
| <b>File</b>                                                                  | 專案中的 .java 原始碼檔案，為 Project 的組成元素。每個 Violation 需關聯到特定的 File 以定位違規位置。                 |
| <b>ViolationReport</b>                                                       | 單次檢查的完整報告，為 ComplianceCheck 的產出物。彙整所有 Violation 並提供統計摘要，供 Developer 或 CI 系統判讀結果。    |
| <b>Violation</b>                                                             | 單筆架構違規記錄，記載哪個 File 違反了哪條 ComplianceRule。為 ViolationReport 的明細項目。                    |
| <b>Suppression</b>                                                           | 開發者標記「已知且接受」的違規抑制記錄，對應 UC-04 的功能。讓團隊可以在不修改程式碼的情況下，將特定違規排除於報告之外。                     |

排除的候選概念與理由：

| Excluded Candidate                          | Reason for Exclusion                                                      |
|---------------------------------------------|---------------------------------------------------------------------------|
| <b>CompilationUnit /<br/>AST</b>            | 這是 JavaParser 函式庫的軟體產物（software artifact），屬於實作層面的概念，不應出現在 Domain Model 中。 |
| <b>ProfileLoader /<br/>ProfileValidator</b> | 這些是軟體類別（software classes），負責解析與驗證 Profile 的內部行為，非領域中的真實概念。                |
| <b>ConsoleReport /<br/>JSONReport</b>       | Console 與 JSON 是 ViolationReport 的輸出格式（SF-04），屬於呈現方式而非領域概念，不需獨立建模。        |
| <b>Developer /<br/>CISystem</b>             | 這些是外部 Actor，不屬於系統內部的領域概念。                                                 |
| <b>ExitCode</b>                             | exit code 是數值型態，適合作為 ComplianceCheck 的屬性而非                                |

| Excluded Candidate                        | Reason for Exclusion                                                                                             |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------|
|                                           | 獨立的概念類別。                                                                                                         |
| <b>ProfileTemplate</b>                    | UC-02 產生的 YAML 模板是 StyleProfile 的輸出／呈現形式（與被排除的 ConsoleReport／JSONReport 同類），屬實作層產物，非領域概念。                        |
| <b>ValidationResult / ValidationError</b> | UC-02 驗證流程的產出。iteration I 已將負責解析與驗證 Profile 的 ProfileValidator 列為軟體類別，其產出同屬實作層，不需建模為領域概念。                        |
| <b>Suggestion（升級為類別）</b>                  | UC-03 的修正建議在本領域僅為一段文字，無自身結構、身份與關聯。依「純資料型別值即屬性」原則，應為 Violation 的屬性 suggestion [0..1]；升級為獨立類別屬過度建模（over-modeling）。 |
| <b>InteractiveSession / FixDecision</b>   | UC-03 的 [y/n/q] 逐條互動為控制流程，非「需要記憶的事物」；其結果（寫入 Suppression 或保留 Violation）已由既有概念建模，無須另立會話類別。                         |

## 2.2 Associations

關聯的建立依據「需要記憶的關係」原則（need-to-remember associations）。下表列出各關聯的名稱、對應數量與建立理由：

| Association                                     | Cardinality | Rationale                              |
|-------------------------------------------------|-------------|----------------------------------------|
| <b>ComplianceCheck Uses StyleProfile</b>        | * — 1       | 每次檢查使用一份 Profile；同一份 Profile 可被多次檢查使用。 |
| <b>ComplianceCheck Scans Project</b>            | * — 1       | 每次檢查掃描一個專案；同一專案可被多次檢查。                 |
| <b>ComplianceCheck Produces ViolationReport</b> | 1 — 1       | 每次檢查產出一份報告，一對一關係。                      |
| <b>ComplianceCheck Reads Suppression</b>        | 1 — 0..*    | 檢查時讀取已存在的 Suppression 記錄來過濾結果。         |
| <b>StyleProfile Contains ComplianceRule</b>     | 1 — 1..*    | 一份 Profile 至少包含一條規則，為組合關係。             |
| <b>Project Contains File</b>                    | 1 — 1..*    | 專案包含一至多個 Java 檔案，為組合關係。                |
| <b>ViolationReport Records Violation</b>        | 1 — 0..*    | 報告記錄零至多筆違規，為報告的明細項目。                   |
| <b>Violation Violates ComplianceRule</b>        | * — 1       | 每筆違規對應一條被違反的規則。                        |



| Association                        | Cardinality | Rationale            |
|------------------------------------|-------------|----------------------|
| Violation Found-in File            | * — 1       | 每筆違規發生在特定的 Java 檔案中。 |
| Suppression Targets ComplianceRule | * — 1       | 每筆抑制記錄對應一條特定規則。      |

此外，NamingRule、DependencyRule、SupertypeRule、PackageRule 為 ComplianceRule 的泛化（generalization）子類型，以空心三角形箭頭表示繼承關係。

## 2.3 Attributes

屬性的選擇遵循「在 Domain Model 中聚焦於資料型別屬性」的原則，避免將複雜概念當作屬性、避免以外鍵（foreign key）代替關聯。各概念類別之屬性整理如下：

| Class           | Attributes                                                                   |
|-----------------|------------------------------------------------------------------------------|
| ComplianceCheck | dateTime（檢查執行時間）、exitCode（結果碼：0=pass, 1=violations, 2=error）                 |
| StyleProfile    | name（Profile 名稱，用於識別）                                                        |
| ComplianceRule  | id（規則識別碼）、description（規則說明文字）                                                |
| NamingRule      | classNamePattern（類別命名的正規表達式 pattern）                                         |
| DependencyRule  | sourcePackage（來源套件 pattern）、targetPackage（目標套件 pattern）、isAllowed（是否允許此依賴方向） |
| SupertypeRule   | targetPackage（適用的套件 pattern）、requiredSupertype（要求繼承或實作的型別名稱）                 |
| PackageRule     | packagePattern（套件結構的萬用字元 pattern）                                            |
| Project         | rootPath（專案根目錄路徑）                                                            |
| File            | filePath（檔案路徑）                                                               |
| ViolationReport | checkedFileCount（已檢查檔案數）、violationCount（違規總數）、suppressedCount（已抑制數量）         |
| Violation       | lineNumber（發生行號）、message（違規說明）、<br>suggestion（針對該筆違規的修正建議，可選 [0..1]）         |
| Suppression     | filePath（被抑制違規所在檔案路徑）、lineNumber（被抑制違規所在行號）、reason（抑制原因）、timestamp（抑制時間戳記）   |

## 3. Revision Based on TA Comments, Midterm Review, and Findings

本次作業進入 Iteration II。以下修訂依據期中 review 與自我檢視進行。本文件之新增／修改處皆以黃色 highlight 標示。

1. Use Case 選擇：選定 UC-02（Define Style Profile）與 UC-03（Check with Interactive Fix Suggestions）為 Iteration II 要實作之 use cases——兩者於期中（Iteration I）未 demo、規劃於期末 demo，且自 HW2 起即以 fully dressed 格式定義。其餘 UC-01／UC-04／UC-05 已於 Iteration I 實作。

2. Domain Model：依 UC-03「逐條顯示修正建議」之需求，於 Violation 新增 suggestion [0..1] 屬性。修正建議係針對「單筆違規」而非「規則」，故置於 Violation；且其為純文字值、無自身結構與關聯。message 與 suggestion 語意切分：message 為錯誤原因與位置，suggestion 為可選之修法。

3. Bad Conceptual Classes：經分析，ProfileTemplate（輸出形式）、ValidationResult／ValidationError（軟體產物）、Suggestion（升級為類別，屬過度建模）、InteractiveSession／FixDecision（控制流程）皆不應納入領域模型，已於 §2.1 明列並說明理由。

4. 模型合併（Combine with Iteration I）：Iteration I 之 12 個概念類別、10 條關聯與既有屬性全數保留不變；Iteration II 對領域模型之唯一結構變更為新增 Violation.suggestion 屬性（未新增概念類別與關聯）。§2 之領域類別圖即為兩迭代之合併模型。

5. 文件結構：依 HW6 規定，本次移除 HW5 之 Design、Implementation Class Model、Programming、Unit Testing 四章節，留待 HW7（Iteration II 之設計與實作）重新提交。

## 4. Measurement

| Student ID | Name | HW   | Date                    | Duration |
|------------|------|------|-------------------------|----------|
| 113598009  | 李俊威  | HW#1 | 115/03/03 15:00 ~ 18:00 | 3H       |
|            |      | HW#2 | 115/03/12 15:00 ~ 21:00 | 6H       |
|            |      | HW#3 | 115/04/01 14:00 ~ 20:00 | 6H       |
|            |      | HW#4 | 115/04/17 14:00 ~ 18:00 | 4H       |
|            |      |      | 115/04/22 13:00 ~ 17:00 | 4H       |
|            |      | HW#5 | 115/04/29 19:00 ~ 23:30 | 4.5H     |
|            |      |      | 115/05/05 15:00 ~ 23:00 | 8H       |
|            |      | HW#6 | 115/05/26 20:00 ~ 23:30 | 3.5H     |